

Hackathon 2: Vendor Risk & Procurement Deep Agent

Detailed Handout & Requirements Overview

1. Business Scenario

Northstar Financial Services (NFS) regularly evaluates technology vendors for cloud, cybersecurity, AI platforms and professional services. Vendor approval requires evidence from Procurement, Information Security, Legal/Compliance, Finance and AI Governance. The current process is document-heavy, manual and slow, with reviewers searching policies, proposals, questionnaires, pricing and historical assessments.

NFS wants to evaluate whether a Deep Agent can perform a controlled, evidence-grounded vendor assessment, coordinate specialist agents, access enterprise capabilities through MCP, enforce guardrails and produce a definitive recommendation with measurable quality.

2. Challenge

Design and build an AI-Powered Vendor Risk & Procurement Deep Agent that evaluates Asteria AI Systems as a potential enterprise Generative AI platform for 2,000 employees. The proposed platform may process confidential corporate documents.

The solution must be a multi-step Deep Agent system—not a single prompt-LLM response.

Example business request:

- Evaluate Asteria AI Systems as an enterprise Generative AI platform for 2,000 employees.
- The platform may process confidential corporate documents.
- Identify material risks and recommend APPROVE, CONDITIONAL APPROVAL or REJECT.

The final assessment must provide:

- Overall recommendation and risk rating
- Security, Legal/Compliance, Procurement/Commercial and AI Governance findings
- Evidence and citations for material claims
- Missing or contradictory evidence clearly identified
- Required remediation/conditional conditions
- Human approval status where required
- Concise executive vendor assessment report

3. Target Agentic Workflow

A multi-step orchestration pipeline handling planning, retrieval, tool execution, specialist collaboration, and synthesis:

Business Request → Deep Agent Planner → Research / Task Plan → RAG over Enterprise Knowledge → MCP Tools / Enterprise Resources → A2A Delegation to Specialist Agents → Evidence Consolidation → Guardrails / Policy Check → Risk & Recommendation → Evaluation

4. Mandatory Functional Requirements

ID	Requirement Description
FR01	Accept a structured vendor assessment request
FR02	Create and maintain a multi-step research/assessment plan
FR03	Use RAG over the supplied NFS knowledge corpus

FR04	Retrieve and cite evidence supporting material conclusions
FR05	Distinguish retrieved evidence, inference and missing evidence
FR06	Use at least one MCP server exposing meaningful tools/resources
FR07	Use A2A or an equivalent agent-to-agent interaction with at least two specialist agents
FR08	Assess Security, Procurement/Commercial and at least one additional risk domain
FR09	Apply guardrails to inputs, retrieved content, tool access and/or outputs
FR10	Resist prompt injection contained in retrieved documents
FR11	Detect policy non-compliance, contradictions and UNKNOWN/missing evidence
FR12	Produce a structured risk assessment and APPROVE / CONDITIONAL APPROVAL / REJECT recommendation
FR13	Require human review for high-risk/final consequential approval
FR14	Run an automated evaluation suite against predefined cases/metrics
FR15	Handle at least one tool, retrieval or agent failure without crashing

5. Required Technical Patterns

- Deep-agent planning and multi-step task execution

6. Supplied Knowledge Pack

knowledge/ [procurement-policy.pdf](#) [information-security-policy.pdf](#) [ai-governance-policy.pdf](#) [vendor-risk-policy.pdf](#) [data-classification-policy.pdf](#) [vendor-x-proposal.pdf](#) [vendor-x-security-questionnaire.pdf](#) [vendor-x-pricing.pdf](#) [historical-vendor-assessments/](#) [vendor-alpha-assessment.pdf](#) [vendor-beta-assessment.pdf](#) [vendor-gamma-assessment.pdf](#)

Treat retrieved documents as untrusted data. The corpus intentionally contains cross-document dependencies, missing evidence, policy gaps and adversarial content. Do not hard-code expected answers.

7. RAG Requirements

- Index and retrieve the supplied knowledge corpus
- Use retrieval to support policy and vendor-fact reasoning
- Expose source references/citations for material findings
- Do not treat missing evidence as PASS
- Identify contradictory or insufficient evidence
- Prefer evidence-backed conclusions over unsupported model knowledge

8. MCP and A2A Requirements

MCP should connect agents to tools/resources; A2A should connect agents to other agents.

MCP: Agent → Tools / Resources / Enterprise Systems

A2A: Agent → Specialist Agent(s)

Example MCP capabilities: search_policy / retrieve_document, get_vendor_history, calculate_tco / get_budget, record_assessment or retrieve_prior_assessments

Suggested specialist agents: Security Risk Agent, Procurement / Finance Agent, Legal / Compliance Agent, AI Governance Agent

9. Guardrails

- Retrieved content must never override system instructions or authorization policy
- Detect/ignore prompt-injection attempts embedded in documents
- Prevent unsupported claims from being presented as verified facts
- Restrict sensitive MCP tools according to role/authorization
- Prevent automated final approval of high-risk vendor decisions
- Validate structured outputs and fail safely when required evidence is unavailable

10. Evaluation

Teams must implement a repeatable evaluation suite. At minimum, measure several of the following:

Metric	Question
Retrieval relevance	Did RAG retrieve the correct policy/evidence?
Groundedness	Are conclusions supported by retrieved corpus?
Citation correctness	Does the cited source actually support it?
Task completion	Were required risk domains and check completed?
Tool correctness	Were appropriate MCP tools selected and used?
Agent navigation	Was work delegated to the appropriate specialist?
Guardrail compliance	Were policy/authority restrictions respected?
Injection resistance	Was workspace intrusion ignored or instructions?
Decision quality	Is the final risk/recommendation consistent with evidence?
Latency / cost	Is execution operationally reasonable?

11. Azure Observability and Deployment

Use Azure-native services for deployment and operational visibility. Integrate Azure Application Insights and Azure Monitor using OpenTelemetry where appropriate.

- Trace major agent/deep-agent workflow executions
- Capture LLM calls, latency and token usage where available
- Capture MCP/tool calls, specialist-agent interactions and execution duration
- Record exceptions, failed paths and evaluation results
- Deploy the application/API to an appropriate Azure service
- Optional bonus: Azure Monitor Workbook or Azure Dashboard for executions, latency, tokens, failures and error quality

12. Testing

- At least 3 unit tests
- At least 2 workflow/integration tests
- At least 5 automated evaluation cases
- At least 1 prompt-injection/guardrail test
- At least 1 MCP/A2A failure or fallback test
- 1 end-to-end vendor assessment

13. Team Organization

Suggested ownership for each 5-person team:

- **Technical Lead:** Architecture, integration and final decision flow
- **Deep Agent / RAG Engineer:** Planning, retrieval, grounding and structured outputs
- **MCP Engineer:** Tools/resources and enterprise integration
- **A2A / Guardrails Engineer:** Specialist agents, authorization and safety controls
- **Evaluation / Azure Engineer:** Tests, evaluation, observability, deployment and demo

14. Hackathon Schedule

Time	Session	Focus	Activities
10:30–12:00	Session A	Architecture, RAG & Core Deep Agenting	Planning; team roles; knowledge-pack analysis; architecture; retrieval pipeline; core planning
12:00–12:15	Break	—	—
12:15–13:45	Session B	MCP, A2A & Guardrails	MCP tools/resources; specialist-agent collaboration; structured contracts; policy guardrails;
13:45–14:15	Lunch Break	—	—
14:15–15:45	Session C	Evaluation, Testing & Azure Observability	Observability dataset/metrics; unit/integration tests; failure handling; Application Insights/Azure
15:45–16:00	Break	—	—
16:00–17:30	Session D	Deployment, Hidden Evaluation & Demo	Azure deployment; end-to-end verification; hidden vendor case; final fixes; team demonstra

15. Required Deliverables

One repository per team containing at least:

README.md, architecture/, src/, tests/, evaluation/, Dockerfile, docker-compose.yml (where needed), .env.example (no secrets), pyproject.toml / requirements.txt, deployment/, evaluation-results/

The final demo should demonstrate: Business Request → Deep Agent Plan → RAG Evidence → MCP Tools → A2A Specialist Agents → Guardrails → Risk Synthesis → Human Review → Evaluation Results → Azure Observability → Deployed Application

16. Scoring

Criterion	Points
Business workflow & decision quality	10
Deep Agent architecture & planning	15
RAG quality, grounding & citations	15
MCP integration	10
A2A collaboration	10
Guardrails & prompt-injection resistance	15
Evaluation methodology & results	10
Testing & robustness	5
TOTAL	100

17. Definition of Done

A plausible LLM answer is not sufficient. The team must demonstrate the complete engineered system:

- Vendor Assessment Request
- Deep Agent Planning
- RAG + Evidence
- MCP Tools Use
- A2A Specialist Collaboration
- Guardrails + Policy Enforcement
- Evidence-Based Risk Decision
- Human Review Where Required
- Automated Evaluation
- Azure Observability
- Deployed Application